



Universidad de Oriente

Núcleo de Monagas

Escuela de Ingeniería y Ciencias Aplicadas

Departamento de Ingeniería de Sistemas

Estructura de Datos (072-3123)

Unidad I.

Representación de los datos en memoria

MSc. Anibal Fariñas

Maturín, abril de 2026

ÍNDICE

1. Fundamentos de la memoria en un computador

- 1.1. Memoria RAM: celda, dirección y contenido.
- 1.2. Memoria estática vs. dinámica (definición, ciclo de vida, ejemplos).
- 1.3. Segmentos de un programa en ejecución: código, datos, pila y montón.

2. Tipos de datos: el molde para la información

- 2.1. Tipos primitivos (enteros, reales, caracteres, lógicos).
- 2.2. Tipos compuestos: arrays, cadenas, estructuras (struct), uniones (union), record en Java.
- 2.3. Comparativa: estructura vs. unión vs. array.

3. Punteros: el arte de direccionar memoria

- 3.1. Concepto de puntero: variable que guarda direcciones.
- 3.2. Declaración, inicialización y desreferencia.
- 3.3. Aritmética de punteros (avance, retroceso, diferencia).
- 3.4. Punteros a estructuras (operador ->).

4. Gestión dinámica de memoria

- 4.1. Memoria en el montón (heap): new y delete en C++.
- 4.2. Analogía con Java (new) y Python (asignación automática).
- 4.3. Constructores y destructores: inicialización y limpieza de recursos.

5. Representación interna en memoria

- 5.1. Formato de almacenamiento: endianness (little-endian vs big-endian).
- 5.2. Alineación de datos (padding) y empaquetamiento (pack).

6. Casos prácticos integradores

- Caso 1: Simulación de una libreta de contactos con memoria dinámica (C++).
- Caso 2: Comparación de rendimiento entre array estático y lista dinámica (Python).

7. Ejercicios prácticos y propuestos

8. Glosario

9. Referencias bibliográficas

1. Fundamentos de la memoria en un computador

1.1. Memoria RAM: celda, dirección y contenido

Definición de memoria del computador

La **memoria del computador** es el componente físico donde se almacenan temporal o permanentemente los datos, las instrucciones de los programas y los resultados intermedios que la CPU (Unidad Central de Procesamiento) necesita para ejecutar sus operaciones. Funciona como un espacio de trabajo accesible por dirección, permitiendo leer y escribir información a gran velocidad.

Analogía general: La memoria es como un escritorio donde el procesador (la persona que trabaja) coloca los papeles (datos) que necesita en este momento. Los papeles que no se usan ahora se guardan en un archivador (disco duro). Cuanto más grande y organizado sea el escritorio, más tareas puede manejar simultáneamente.

Jerarquía de memoria

Para equilibrar velocidad, costo y capacidad, los computadores organizan la memoria en una jerarquía:

Nivel	Ejemplo	Velocidad (acceso)	Capacidad típica	Persistencia
Registros de CPU	Registros internos	< 1 ns	Bytes – KB	Volátil
Caché (L1, L2, L3)	SRAM	1–10 ns	KB – MB	Volátil
Memoria principal	RAM (DRAM)	10–100 ns	GB	Volátil
Almacenamiento masivo	SSD / HDD	0.1–10 ms	TB	No volátil

Tipos de memoria principal

- **RAM (Random Access Memory):** Lectura y escritura, volátil (pierde datos al apagar). Se subdivide en:
 - **DRAM (Dinámica):** Necesita refresco periódico; más lenta pero económica. Es la más común en computadores.
 - **SRAM (Estática):** Más rápida, no necesita refresco; usada en caché.
- **ROM (Read Only Memory):** Solo lectura, no volátil. Almacena firmware (BIOS, UEFI).

Celda, dirección y contenido (ampliación)

Como se mencionó, la RAM se organiza como un arreglo de celdas. Cada celda almacena un byte (8 bits) y tiene una dirección única (un número entero). El procesador puede acceder a cualquier celda en el mismo tiempo (acceso aleatorio).

- **Contenido de una celda:** Secuencia de bits (0 y 1) que representa un carácter, parte de un número o una instrucción.
- **Dirección:** Número que identifica la posición. En arquitecturas modernas, las direcciones son de 32 o 64 bits (espacio de direcciones de hasta 2^{32} o 2^{64} bytes).

Ejemplo en ensamblador (conceptual):

`MOV AL, [0x7FFD3C2A]` ; Lee el byte en la dirección 0x7FFD3C2A y lo coloca en el registro AL

Representación de datos en memoria

Los datos se almacenan en memoria según su tamaño y formato:

- **Entero pequeño (1 byte):** Ocupa una celda.
- **Entero de 4 bytes (int):** Ocupa 4 celdas consecutivas.
- **Cadena de texto:** Ocupa tantas celdas como caracteres (más el terminador nulo en C/C++).

Ejemplo visual (direcciones hipotéticas):

Dirección	Contenido (hex)
0x1000	0x48 ('H')
0x1001	0x65 ('e')
0x1002	0x6C ('l')
0x1003	0x6C ('l')
0x1004	0x6F ('o')
0x1005	0x00 (terminador)

Capacidad y unidades de medida

- 1 byte = 8 bits
- 1 KB (kilobyte) = 1024 bytes $\approx 10^3$
- 1 MB (megabyte) = 1024 KB $\approx 10^6$
- 1 GB (gigabyte) = 1024 MB $\approx 10^9$
- 1 TB (terabyte) = 1024 GB $\approx 10^{12}$

Ejemplo concreto: Un programa que declara un arreglo `int arr[1000]`; en C++ ocupará aproximadamente 4000 bytes ($1000 * 4$) en memoria (más posible padding si está dentro de una estructura).

Relación con la programación de alto nivel

Cuando declaras una variable en C++, Java o Python, el sistema operativo y el compilador deciden en qué dirección de memoria se almacenará y cuántos bytes ocupará. El programador, mediante punteros (C/C++) o referencias (Java/Python), puede manipular esas direcciones.

Ejemplo en C++ que muestra direcciones:

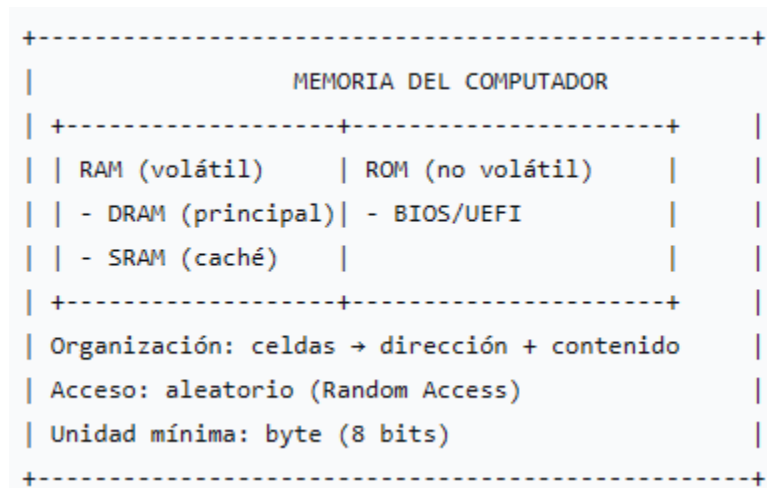
```
#include <iostream>
int main() {
    int a = 42;
    std::cout << "Valor: " << a << std::endl;
    std::cout << "Dirección: " << &a << std::endl; // imprime un número
    hexadecimal
    return 0;}
```

Ejemplo en Python (obtener representación de la dirección):

```
a = 42
print(hex(id(a))) # imprime la dirección virtual del objeto entero
```

Conceptos adicionales importantes

- **Memoria virtual:** El sistema operativo crea la ilusión de que el programa tiene un espacio de direcciones continuo y enorme, mapeando direcciones virtuales a direcciones físicas reales mediante la unidad de gestión de memoria (MMU).
- **Endianness:** Orden en que se almacenan los bytes de una palabra.
- **Alineación:** Restricción de direcciones para ciertos tipos.



La memoria RAM (Random Access Memory) es un conjunto de celdas, cada una con una dirección única (un número entero, por ejemplo, 0x7FFD3C2A) y un contenido (un byte, típicamente 8 bits). El procesador lee o escribe datos especificando la dirección.

Analogía: La memoria es como un edificio de apartamentos. Cada apartamento tiene un número de dirección y dentro vive un byte de información.

1.2. Memoria estática vs. dinámica

Característica	Memoria estática	Memoria dinámica (Montón/Heap)
Asignación	En tiempo de compilación.	En tiempo de ejecución.

Característica	Memoria estática	Memoria dinámica (Montón/Heap)
Tamaño	Fijo (debe conocerse antes).	Variable (se pide y libera bajo demanda).
Ubicación	Segmento de datos o pila.	Segmento del montón (heap).
Ciclo de vida	Toda la ejecución del programa o hasta que termina el bloque (variables locales).	Controlado por el programador (hasta que se libera).
Ejemplo en C++	<code>int arr[100];</code>	<code>int* arr = new int[100];</code>
Ejemplo en Java	<code>int[] arr = new int[100];</code> (el objeto está en heap, pero la referencia es estática si es local)	<code>new</code> siempre asigna en heap.
Ejemplo en Python	<code>x = 5</code> (el objeto 5 está en heap, pero la referencia está en un entorno estático)	Todo objeto creado con <code>list()</code> , <code>dict()</code> , etc. está en heap.

Concepto clave:

La **memoria estática** no se libera hasta que la variable sale de ámbito (stack) o termina el programa (datos globales).

La **memoria dinámica** persiste hasta que se llame a delete (C++) o el recolector de basura la elimine (Java/Python).

Tabla comparativa: Memoria estática vs Memoria dinámica

Característica	Memoria Estática	Memoria Dinámica
Momento de asignación	En tiempo de compilación (el compilador fija el tamaño y la ubicación).	En tiempo de ejecución (el programa solicita memoria cuando la necesita).
Tamaño	Fijo e inmutable durante toda la ejecución.	Variable; puede crecer o reducirse bajo demanda.

Característica	Memoria Estática	Memoria Dinámica
Ubicación en memoria	Segmento de datos (globales/estáticas) o pila (stack) (locales).	Segmento del montón (heap).
Ciclo de vida	- Variables globales/estáticas: todo el programa. - Variables locales (stack): hasta que termina el bloque/función.	Desde que se asigna con new/malloc hasta que se libera explícitamente (delete/free) o el recolector de basura la reclama.
Liberación	Automática (el compilador genera código para liberar la pila al salir del ámbito; los datos globales se liberan al terminar el programa).	Manual en C/C++ (delete), automática en Java/Python (recolector de basura).
Velocidad de acceso	Muy rápida (acceso directo, a menudo en caché).	Más lenta (indirección a través de punteros, posible fragmentación).
Riesgos típicos	Desbordamiento de pila (stack overflow), desperdicio de memoria si se asigna más de lo necesario.	Fugas de memoria (no liberar), punteros colgantes (acceder después de liberar), fragmentación.
Flexibilidad	Baja: no se puede cambiar el tamaño en tiempo de ejecución.	Alta: se adapta a las necesidades del programa (listas, árboles, etc.).

Ventajas y desventajas detalladas

Memoria Estática

Ventajas

1. **Simplicidad** – El programador no se preocupa por asignar/liberar memoria; el compilador lo maneja.
2. **Rendimiento** – El acceso es directo y muy rápido (sin indirección de punteros ni sobrecarga de gestión de heap).
3. **Sin fragmentación** – No hay riesgo de fragmentación de memoria porque los bloques son fijos y se asignan de manera contigua.

4. **Seguridad** – No hay fugas de memoria ni punteros colgantes (en el stack, las variables se destruyen automáticamente).

Desventajas

1. **Tamaño fijo** – Una vez declarado, no puede crecer ni reducirse. Si necesita más espacio, no hay solución (a menos que rediseñe el programa).
2. **Desperdicio o insuficiencia** – Si se reserva un array de 1000 elementos pero solo se usan 10, se desperdicia memoria. Si se necesitan 1001, el programa falla.
3. **Vida útil limitada** – Las variables locales (stack) dejan de existir al salir de la función; no se pueden retornar punteros a dichas variables (son inválidas fuera del ámbito).
4. **Limitación de tamaño** – La pila tiene un tamaño fijo (típicamente 1-8 MB); declarar grandes arrays locales puede causar *stack overflow*.

Memoria Dinámica

Ventajas

1. **Flexibilidad total** – El tamaño puede determinarse en tiempo de ejecución, incluso basado en la entrada del usuario o la carga de datos.
2. **Eficiencia en memoria** – Se asigna exactamente lo necesario; se puede redimensionar (en C++ con `realloc`, en Java/Python con colecciones dinámicas).
3. **Persistencia controlada** – La memoria puede mantenerse viva más allá del ámbito de una función (útil para retornar estructuras complejas).
4. **Permite estructuras de datos avanzadas** – Listas enlazadas, árboles, grafos, etc., que necesitan nodos creados dinámicamente.

Desventajas

1. **Complejidad de gestión (C/C++)** – El programador debe recordar liberar cada bloque con `delete`; olvidarlo produce **fugas de memoria**.
2. **Riesgo de punteros colgantes** – Si se libera memoria y luego se accede a ella mediante otro puntero, el comportamiento es indefinido (crash o datos corruptos).

3. **Fragmentación** – Con el tiempo, el heap puede llenarse de pequeños huecos no contiguos, haciendo que una asignación grande falle aunque haya memoria libre total suficiente.
4. **Sobrecarga de rendimiento** – Asignar y liberar memoria dinámica (especialmente en C/C++ con new/delete) tiene costo; acceder a través de punteros puede implicar una indirección adicional.
5. **Recolector de basura (Java/Python)** – Aunque elimina las fugas, introduce pausas impredecibles (GC) y mayor consumo de memoria.

Ejemplo concreto comparativo (C++)

```
// MEMORIA ESTÁTICA (stack)
void funcionEstatica() {
    int arr[1000];          // Fijo: siempre 1000 enteros, se asigna en
stack
    // ... usar arr
} // al salir, arr se libera automáticamente

// MEMORIA DINÁMICA (heap)
void funcionDinamica(int n) {
    int* arr = new int[n]; // Tamaño variable según n, se asigna en heap
    // ... usar arr
    delete[] arr;          // Obligatorio liberar (si no, fuga)
}
```

Diferencia crítica: En `funcionEstatica`, `n` debe ser conocido en tiempo de compilación (en C++ estándar, los arrays de tamaño variable no están permitidos en stack, aunque algunos compiladores los admiten como extensión). En `funcionDinamica`, `n` puede ser cualquier valor calculado en tiempo de ejecución.

Analogía para entender la diferencia

- **Memoria estática (stack):** Es como tener un cajón de escritorio fijo con compartimentos numerados. No puedes cambiar el número de compartimentos, pero abrirlos y cerrarlos es ultrarrápido. Si necesitas guardar más cosas de las que caben, no puedes.
- **Memoria dinámica (heap):** Es como alquilar almacenes bajo demanda. Puedes pedir un almacén de cualquier tamaño, devolverlo cuando ya no lo necesites, y pedir

otro más grande. Es más lento y debes recordar devolver la llave (liberar), o el almacén quedará ocupado para siempre (fuga).

Cuándo usar cada una

Situación	Recomendación
Array de tamaño conocido y pequeño (ej. fijo 100 elementos).	Estática (stack)
Variable local cuyo tamaño se conoce en compilación y no excede los pocos KB.	Estática (stack)
Datos que deben persistir después de que la función termine (ej. retornar una lista).	Dinámica (heap)
Tamaño determinado por entrada del usuario o archivo de datos.	Dinámica (heap)
Estructuras recursivas profundas (árboles, grafos).	Dinámica (heap)
Necesidad de redimensionar una colección (ej. añadir elementos sin límite).	Dinámica (heap)
Programación en Java/Python (todo objeto está en heap; las variables locales solo son referencias).	Dinámica (por diseño del lenguaje)

Nota sobre Java y Python

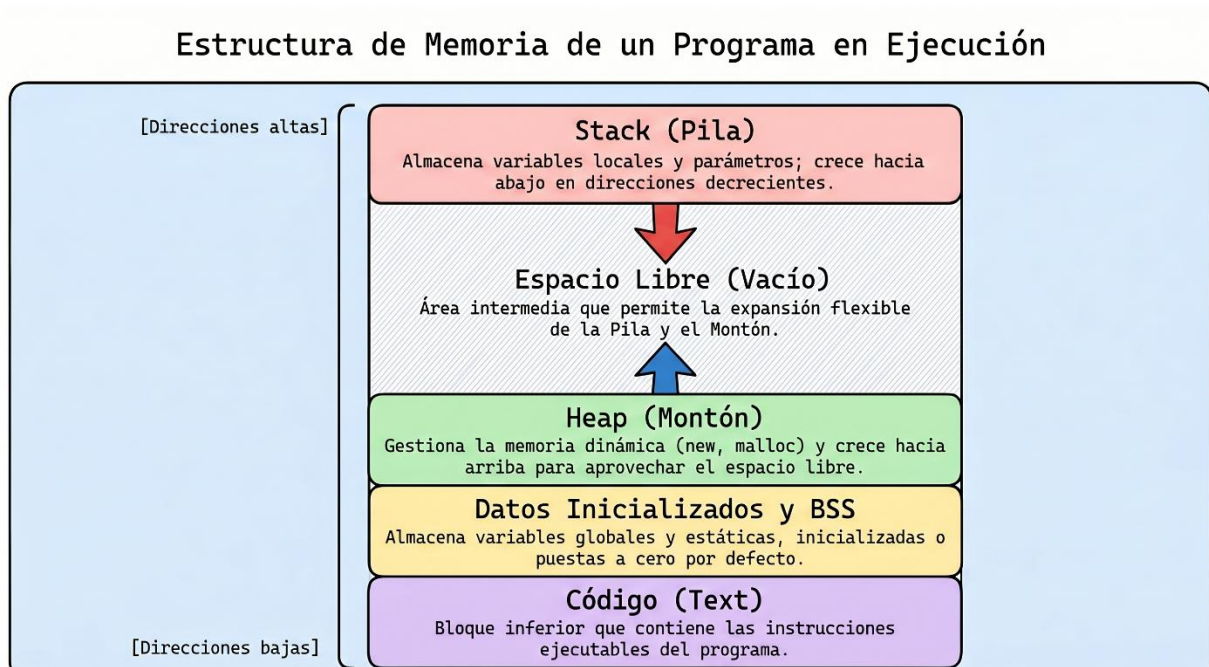
En estos lenguajes, todos los objetos viven en el heap (memoria dinámica). Las variables locales que son tipos primitivos (int, float, bool) pueden estar en el stack, pero los objetos siempre están en heap. El recolector de basura (GC) libera automáticamente la memoria dinámica cuando ya no hay referencias. Esto elimina las fugas, pero introduce:

- **Pausas eventuales** (stop-the-world).
- **Mayor consumo de memoria** (el GC puede posponer la liberación).

Por tanto, aunque el programador no usa delete, sigue siendo memoria dinámica con sus ventajas (flexibilidad) y desventajas (sobrecarga, fragmentación manejada por el VM).

1.3. Segmentos de un programa en ejecución

Cuando un programa se ejecuta, el sistema operativo organiza su espacio de direcciones en segmentos:



- **Segmento de código (Text):** Contiene las instrucciones binarias. Es de solo lectura.
- **Segmento de datos (Data/BSS):** Variables globales y estáticas. Data guarda las inicializadas, BSS las que se inicializan a cero.
- **Pila (Stack):** Administración de llamadas a funciones. Cada llamada crea un marco de pila con variables locales. Es automática (LIFO).
- **Montón (Heap):** Región para asignación dinámica. El programador controla su uso.

Ejemplo concreto (C++):

```
int global = 10;           // Segmento de datos inicializados
static int estatica;       // Segmento BSS (se inicializa a 0)

void funcion() {
    int local;              // Pila (stack)
    int* dinamico = new int(5); // `dinamico` está en pila, pero el
    entero 5 está en heap
    delete dinamico;
```

```
}
```

2. Tipos de datos: el molde para la información

2.1. Tipos primitivos

Son los bloques básicos que el procesador entiende directamente. Ocupan un tamaño fijo.

C++/Java	Python (tipos implícitos)	Tamaño típico (C++)	Representación aproximada
int	int (arbitrario)	4 bytes	2^{32} valores, complemento a 2
float	float (por float())	4 bytes	IEEE 754: 1 bit signo, 8 exponente, 23 mantisa
double	float (por defecto)	8 bytes	IEEE 754: 1, 11, 52
char	str de un carácter	1 byte	Código ASCII / Unicode (UTF-8)
bool	bool	1 byte	0 = false, !=0 = true

2.2. Tipos compuestos

Son agregaciones de tipos primitivos u otros compuestos.

Arrays: Colección homogénea de elementos contiguos en memoria.

```
// C++
```

```
int arr[5] = {1,2,3,4,5}; // arr[0] dirección base, arr[1] = base + 1*tamaño_int
```

Estructuras (struct en C/C++, class en Java/C#): Agrupa datos heterogéneos. Cada miembro tiene su propio espacio.

```
struct Persona {
    char nombre[50];
    int edad;
    float altura;
}; // Ocupa: 50+4+4 = 58 bytes + posible padding
```

Uniones (union): Todos los miembros comparten la misma dirección de memoria. Solo un miembro activo a la vez. Útil para ahorrar memoria o interpretar un mismo dato de distintas formas.

cpp

```
union Dato {
    int i;
    float f;
    char c;
}; // Ocupa 4 bytes (el mayor de los miembros). Si asignas i=100, luego f
será un número extraño.
```

record en Java (a partir de Java 14/16)

Un record es una forma especial de clase en Java diseñada para actuar como portador transparente de datos inmutables. Declara de manera concisa los campos, y el compilador genera automáticamente:

- Constructor con todos los campos.
- Métodos de acceso (getters con el mismo nombre del campo).
- equals(), hashCode() y toString() basados en los campos.
- Los campos son final (inmutables después de la construcción).

Sintaxis básica:

```
record Persona(String nombre, int edad, float altura) { }
```

Equivalencia con una clase tradicional:

El record anterior reemplaza decenas de líneas de código que incluirían campos privados, constructor, getters, equals, hashCode y toString.

Características clave del record:

Característica	Explicación
Inmutabilidad	Todos los campos son final; una vez creado, el objeto no puede cambiar.
Transparencia	Los métodos de acceso tienen el mismo nombre que los campos (sin get).
Agregación de datos	Ideal para agrupar datos heterogéneos (similar a struct en C o dataclass en Python).

Característica	Explicación
Sin herencia	Un record no puede extender otra clase (excepto implícitamente Record), pero puede implementar interfaces.
Compacto	Una sola línea define el tipo; el compilador genera el resto.

Comparación con struct de C/C++ y class de Java:

Aspecto	struct (C/C++)	class (Java)	record (Java)
Mutabilidad	Por defecto mutable (puede cambiar)	Mutable o inmutable según diseño	Inmutable (campos final)
Métodos generados	Ninguno (solo datos)	Deben escribirse explícitamente	Constructor, getters, equals, hashCode, to String automáticos
Sobrecarga de código	Baja	Alta (para clases CRUD simples)	Muy baja
Uso típico	Agregados ligeros, POD (Plain Old Data)	Objetos con comportamiento completo	DTO (Data Transfer Object), valores inmutables, claves de mapas
Herencia	Sí (en C++)	Sí	No (excepto interfaces)

Ejemplo completo en Java (comentado):

```
// Definición de un record para representar una persona
record Persona(String nombre, int edad, float altura) { }

public class Main {
    public static void main(String[] args) {
        // Crear instancias usando el constructor generado
        automáticamente
        Persona p1 = new Persona("Ana", 25, 1.65f);
        Persona p2 = new Persona("Luis", 30, 1.80f);

        // Acceso a los campos (sin prefijo 'get')
        System.out.println(p1.nombre()); // Ana
        System.out.println(p1.edad());   // 25
        System.out.println(p1.altura()); // 1.65
    }
}
```

```

        // toString() generado
        System.out.println(p1); // Persona[nombre=Ana, edad=25,
altura=1.65]

        // equals() y hashCode() basados en todos los campos
        Persona p3 = new Persona("Ana", 25, 1.65f);
        System.out.println(p1.equals(p3)); // true

        // Inmutabilidad: no hay setters, los campos son final
        // p1.edad = 26; // Error de compilación
    }
}

```

¿Cuándo usar record en lugar de class o struct?

- Usa record cuando necesites una agregación simple de datos, sin comportamiento complejo, y quieras inmutabilidad automática (ej. DTO, resultados de consultas, claves compuestas).
- Usa class cuando necesites mutabilidad, herencia, o lógica de negocio que modifique el estado interno.
- Comparación con struct de C++: record es similar pero inmutable por diseño. En C++ puedes lograr inmutabilidad con const miembros, pero no es automático.

Equivalente en otros lenguajes:

- **Python:** @dataclass(frozen=True) (inmutable) o dataclass normal.

```

from dataclasses import dataclass
@dataclass(frozen=True)
class Persona:
    nombre: str
    edad: int
    altura: float

```

- **C#:** record class (C# 9) – similar a Java.

```

public record Persona(string Nombre, int Edad, float Altura);

```

- **C++ (moderno):** No hay equivalente directo; se puede emular con struct + constructor constexpr y métodos const.

Ejemplo práctico: Uso de record como clave en un HashMap

Los record generan hashCode() y equals() basados en todos los campos, por lo que son ideales como claves.

```
record Coordenada(int x, int y) { }
```

```
Map<Coordenada, String> mapa = new HashMap<>();  
mapa.put(new Coordenada(10, 20), "Origen");  
System.out.println(mapa.get(new Coordenada(10, 20))); // "Origen"
```

Limitaciones a tener en cuenta:

- No pueden extender otras clases (pero sí implementar interfaces).
- Son implícitamente final; no se puede crear una subclase de un record.
- No pueden tener campos de instancia adicionales fuera de los declarados en la cabecera.
- Pueden declarar métodos estáticos, constructores compactos o personalizados, y métodos de instancia adicionales.

```
record Circulo(double radio) {  
    // Constructor compacto para validación  
    public Circulo {  
        if (radio <= 0) throw new IllegalArgumentException("Radio  
positivo");  
    }  
    // Método adicional  
    public double area() { return Math.PI * radio * radio; }  
}
```

Resumen integrador de tipos compuestos

Tipo compuesto	Lenguajes	Características principales
Array	C, C++, Java, Python, etc.	Colección homogénea, tamaño fijo (en C/C++/Java) o dinámico (Python lista). Acceso por índice.
Struct	C/C++	Agrupar datos heterogéneos, mutable, control total de memoria (incluye padding).
Union	C/C++	Todos los miembros comparten memoria, solo uno activo, ahorro extremo.

Tipo compuesto	Lenguajes	Características principales
Class	Java, C++, C#, Python	Agrupar datos + comportamiento, herencia, polimorfismo, encapsulación.
Record	Java (14+), C# (9+)	Agregado simple de datos, inmutable, constructor/getters/equals/hashCode automáticos.

2.3. Comparativa: estructura vs. unión vs. array

Aspecto	Array	Struct	Union
Heterogeneidad	No (todos iguales)	Sí	Sí (pero solo uno a la vez)
Memoria total	suma de todos los elementos	suma de miembros + padding	tamaño del miembro más grande
Acceso simultáneo	a todos los elementos	a todos los miembros	solo el último asignado está válido
Caso de uso	listas, matrices, buffers	registros, objetos ligeros	ahorro extremo o reinterpretación

3. Punteros: el arte de direccionar memoria

3.1. Concepto de puntero

Un puntero es una variable que almacena una dirección de memoria. En lugar de contener un dato directamente (ej. `int x=10`), contiene la dirección donde ese dato se encuentra.

Analogía: La dirección de tu casa (puntero) te permite encontrar la casa (dato). Sin punteros, tendrías que llevar la casa entera contigo.

3.2. Declaración, inicialización y desreferencia

Operación	C/C++	Java (objetos)	Python (todo son referencias)
Declarar	<code>int *ptr;</code>	<code>MiClase obj; (null)</code>	<code>obj = None</code>
Inicializar	<code>ptr = &variable;</code>	<code>obj = new MiClase()</code>	<code>obj = MiClase()</code>
Desreferencia	<code>*ptr = 20;</code>	<code>obj.metodo()</code>	<code>obj.atributo</code>
Obtener dirección	<code>&variable</code>	No disponible	<code>id(variable)</code> (dir. virtual)

Ejemplo en C++ comentado:

```
#include <iostream>
using namespace std;

int main() {
    int numero = 42;           // Variable común
    int *puntero = &numero;    // puntero guarda la dirección de 'numero'

    cout << "Valor de numero: " << numero << endl;           // 42
    cout << "Dirección de numero: " << &numero << endl;      // 0x16fdff2b8
    (ejemplo)
    cout << "Contenido de puntero: " << puntero << endl;    // misma
    dirección
    cout << "Desreferencia de puntero: " << *puntero << endl; // 42

    *puntero = 100;           // Cambia el valor en la dirección apuntada
    cout << "Nuevo valor de numero: " << numero << endl;    // 100
    return 0;
}
```

3.3. Aritmética de punteros

Los punteros se pueden incrementar (++), decrementar (--) y restar entre sí. El avance es en unidades del tamaño del tipo base.

```
int arr[] = {10, 20, 30, 40};
int *p = arr; // p apunta a arr[0]

cout << *p << endl;    // 10
p++;                  // avanza 4 bytes (tamaño de int)
cout << *p << endl;    // 20
p += 2;               // avanza 8 bytes (2 ints)
cout << *p << endl;    // 40 (arr[3])
```

```
int *q = &arr[3];
cout << "Diferencia: " << q - p << endl; // 0 (están en la misma
posición)
```

Aplicación práctica: Recorrer arrays sin usar índices, o implementar iteradores manuales.

3.4. Punteros a estructuras

Acceder a un miembro de una estructura a través de un puntero requiere el operador -> (azúcar sintáctico de (*puntero).miembro).

Ejemplo C++:

```
struct Punto {
    int x;
    int y;
};

int main() {
    Punto p1 = {5, 10};
    Punto *ptr = &p1;

    // Dos formas equivalentes
    (*ptr).x = 8; // desreferencia + acceso
    ptr->y = 20; // operador flecha (más limpio)

    cout << "p1 = (" << p1.x << "," << p1.y << ")" << endl; // (8,20)
    return 0;
}
```

Ejemplo Java (los objetos se usan siempre con referencia implícita):

```
class Punto { int x, y; }
Punto p = new Punto();
p.x = 5; // Java maneja la referencia automáticamente, similar a
puntero.
```

Ejemplo Python (todo es referencia):

```
class Punto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Punto(5, 10)
# Se comporta como un puntero implícito
```

4. Gestión dinámica de memoria

4.1. Memoria en el montón (heap): new y delete en C++

La memoria dinámica se solicita del montón cuando no sabemos de antemano cuánto espacio necesitaremos o queremos que los datos persistan más allá del ámbito de una función.

Operador	Acción
new tipo	Asigna memoria para un objeto del tipo y llama al constructor. Devuelve un puntero.
delete ptr	Libera la memoria apuntada por ptr y llama al destructor.
new tipo[n]	Asigna un arreglo de n elementos. Devuelve puntero al primero.
delete[] ptr	Libera un arreglo dinámico.

Ejemplo completo comentado:

```
#include <iostream>
using namespace std;

int main() {
    // Asignación de un entero dinámico
    int *p = new int(25);    // reserva memoria en heap y guarda 25
    cout << "Valor: " << *p << endl;    // 25
    *p = 30;
    cout << "Modificado: " << *p << endl; // 30
    delete p;                // importante: liberar para evitar fuga de
    memoria
    p = nullptr;             // buena práctica: evitar puntero colgante

    // Asignación de un arreglo dinámico de tamaño variable
    int n;
    cout << "¿Cuántos números? ";
    cin >> n;
    int *arr = new int[n];    // array de n enteros en heap
    for (int i = 0; i < n; ++i)
        arr[i] = i * 10;

    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
    cout << endl;

    delete[] arr;            // liberar arreglo
    return 0;
}
```

4.2. Analogía con Java y Python

- **Java:** Todo objeto se asigna con `new` en el heap. No hay delete; el **Garbage Collector (GC)** libera la memoria automáticamente cuando no hay referencias.

`Integer num = new Integer(10);` // en heap, GC decide cuándo eliminar

- **Python:** Similar a Java, todo es objeto en heap, el GC con conteo de referencias o generacional.

`lista = [1,2,3]` # la lista está en heap; 'lista' es una referencia

4.3. Constructores y destructores

Cuando asignamos objetos dinámicos, C++ permite constructores (inicialización) y destructores (limpieza de recursos).

```
class Recurso {
public:
    Recurso() { cout << "Constructor: adquiriendo recurso\n"; }
    ~Recurso() { cout << "Destructor: liberando recurso\n"; }
};

int main() {
    Recurso *r = new Recurso(); // llama al constructor
    // ... uso
    delete r;                  // llama al destructor automáticamente
    return 0;
}
```

¿Por qué es importante? Si el objeto tiene memoria dinámica interna, el destructor debe liberarla (por ejemplo, un arreglo dentro de la clase). En Java/Python, el destructor no es confiable, se usa `finalize()` (Java) o `__del__` (Python) pero no se recomienda.

5. Representación interna en memoria

5.1. Formato de almacenamiento: endianness

Endianness es el orden en que los bytes de una palabra (multibyte) se almacenan en memoria.

- **Little-endian:** El byte menos significativo va en la dirección más baja. (usado en x86, ARM en modo little)
 - Ej: `int x = 0x12345678` se guarda como 78 56 34 12 en memoria creciente.
- **Big-endian:** El byte más significativo va en la dirección más baja. (usado en redes, algunos procesadores)

- El mismo 0x12345678 se guarda como 12 34 56 78.

Ejemplo para detectar (C++):

```
#include <iostream>
int main() {
    unsigned int num = 1;
    unsigned char *bytePtr = (unsigned char*)&num;
    if (*bytePtr == 1)
        std::cout << "Little-endian\n";
    else
        std::cout << "Big-endian\n";
    return 0;
}
```

5.2. Alineación y empaquetamiento

El compilador a menudo añade *bytes de relleno* (padding) entre los miembros de una estructura para que cada uno esté alineado a una dirección múltiplo de su tamaño (acceso más rápido).

```
struct Ejemplo {
    char c;    // 1 byte
    // 3 bytes de padding (si int necesita alineación a 4)
    int i;     // 4 bytes
    short s;   // 2 bytes
    // 2 bytes de padding al final para alinear la estructura entera a
    // múltiplo de 4
};
// sizeof(Ejemplo) == 12, aunque suma de miembros = 1+4+2=7
```

Empaquetamiento (#pragma pack): Forzar a que no haya padding, ahorrando memoria a costa de rendimiento.

```
#pragma pack(push, 1)
struct Empacado {
    char c;
    int i;
    short s;
};
#pragma pack(pop)
// sizeof(Empacado) == 1+4+2 = 7 (sin padding)
```

Caso real: Transmisión de datos por red o lectura de formatos binarios (BMP, archivos raw) donde el padding rompe la compatibilidad.

6. Casos prácticos integradores

Caso 1: Libreta de contactos con memoria dinámica (C++)

Situación: Un programador debe crear un sistema que permita agregar contactos (nombre, teléfono) sin límite predefinido. Inicialmente usó un array estático de 100 contactos, pero el cliente reportó que se queda corto. Se necesita rediseñar usando memoria dinámica.

Acciones realizadas:

- Se define una estructura Contacto.
- Se usa un puntero Contacto* agenda y un entero capacidad y cantidad.
- Cada vez que se supera la capacidad, se duplica con new[], se copian los datos y se libera la anterior.

Hallazgos:

- El array estático desperdiciaba memoria (si hay 5 contactos, 95 lugares vacíos) o se desbordaba.
- La versión dinámica crece bajo demanda, ajustando la memoria al uso real.

Recomendaciones: Liberar la memoria al final con delete[]. Siempre validar que new no lance excepción (o capturarla).

Resultados: Se implementa una libreta dinámica eficiente. El cliente puede agregar miles de contactos sin desperdicio.

Código simplificado comentado:

```
#include <iostream>
#include <string>
using namespace std;

struct Contacto {
    string nombre;
    string telefono;
};

class AgendaDinamica {
    Contacto* contactos; // puntero a arreglo dinámico
    int capacidad;
    int total;

    void redimensionar(int nuevaCap) {
        Contacto* nuevo = new Contacto[nuevaCap];
        for (int i = 0; i < total; i++)
            nuevo[i] = contactos[i];
        delete[] contactos;
```

```

        contactos = nuevo;
        capacidad = nuevaCap;
        cout << "Capacidad ampliada a " << nuevaCap << endl;
    }

public:
    AgendaDinamica() : capacidad(2), total(0) {
        contactos = new Contacto[capacidad];
    }

    ~AgendaDinamica() { delete[] contactos; }

    void agregar(const string& nom, const string& tel) {
        if (total == capacidad)
            redimensionar(capacidad * 2);
        contactos[total].nombre = nom;
        contactos[total].telefono = tel;
        total++;
    }

    void mostrar() {
        for (int i = 0; i < total; i++)
            cout << i+1 << ". " << contactos[i].nombre << " - " <<
contactos[i].telefono << endl;
    }
};

int main() {
    AgendaDinamica agenda;
    agenda.agregar("Ana", "555-1234");
    agenda.agregar("Luis", "555-5678");
    agenda.agregar("Carlos", "555-9012"); // dispara redimensionamiento
    agenda.mostrar();
    return 0;
}

```

Caso 2: Comparación de rendimiento entre array estático y lista dinámica (Python)

Situación: Un analista de datos necesita procesar un flujo de 10 millones de números pero no sabe cuántos vendrán. Decide comparar dos enfoques: preasignar un array de tamaño fijo (array.array o lista pre-inicializada) vs usar una lista dinámica de Python (que internamente redimensiona).

Acciones realizadas: Se crean dos funciones:

- `usar_lista_dinamica()`: simplemente `lista.append(x)`.
- `usar_array_estatico()`: crear `lista = [0]*tamaño_maximo` y llevar un índice.

Hallazgos: La lista dinámica redimensiona varias veces (copia $O(n)$ cada vez), pero en promedio su costo amortizado es $O(1)$. El array estático requiere conocer el límite superior con antelación; si se subestima, error; si se sobreestima, desperdicio.

Recomendaciones: Para flujos de tamaño desconocido pero grande, usar estructuras dinámicas (listas de Python, `std::vector` en C++, `ArrayList` en Java). Si el tamaño máximo es conocido y fijo, el array estático es más rápido por menor overhead.

Resultados: En el escenario real, la lista dinámica manejó 10M de elementos sin problema, mientras que el array estático con un límite alto usó más memoria innecesaria.

Código de simulación simple:

```
import time
import sys

def dinamico(datos):
    lista = []
    for x in datos:
        lista.append(x)
    return lista

def estatico(datos, max_est):
    if len(datos) > max_est:
        raise ValueError("Desbordamiento")
    lista = [0] * max_est
    for i, x in enumerate(datos):
        lista[i] = x
    return lista[:len(datos)]

# Caso real: generador de 5 millones
muestra = range(5_000_000)

inicio = time.time()
din = dinamico(muestra)
print(f"Lista dinámica: {time.time()-inicio:.2f}s, tamaño final {sys.getsizeof(din)} bytes")

LIMITE = 10_000_000 # sobreestimación
inicio = time.time()
est = estatico(muestra, LIMITE)
print(f"Array estático: {time.time()-inicio:.2f}s, tamaño final {sys.getsizeof(est)} bytes")
```

7. Ejercicios prácticos resueltos y propuestos

Ejercicio resuelto 1 (análisis de memoria)

Enunciado: Dado el siguiente código C++, dibuja el estado de la pila y el montón después de ejecutar cada línea. Identifica fugas de memoria.

```
int* a = new int(10);  
int* b = new int(20);  
a = b;  
delete b;
```

Solución:

1. a apunta a heap (10).
2. b apunta a heap (20).
3. $a = b \rightarrow$ ahora a apunta al mismo heap que b (20). El bloque que contenía 10 queda sin referencia (fuga de memoria).
4. delete b libera el bloque de 20. a ahora es un puntero colgante.
Fuga detectada: el bloque de 10 nunca se liberó.

Ejercicio resuelto 2 (alineación)

Enunciado: En una arquitectura de 32 bits donde cada tipo se alinea a su tamaño, ¿cuánto ocupa struct S { char a; double b; char c; };? ¿Cómo reducirlo con #pragma pack?

Solución:

- Sin packing: char a (1 byte + 7 padding para que double empiece en offset múltiplo de 8), double b (8), char c (1 + 7 padding al final para que la estructura sea múltiplo de 8) $\rightarrow$ total 24 bytes.
- Con #pragma pack(1): $1+8+1 = 10$ bytes.

Ejercicios propuestos

1. **Código en Java:** ¿Dónde se almacenan los siguientes elementos? `int x = 5;` `Integer y = new Integer(10);` `int[] z = new int[3];`
2. **Aritmética de punteros (C++):** Dado `char* p = "ABCD";`, ¿qué valor tiene `*(p+2)` y `p[2]`? ¿Qué ocurre si haces `p++`?

3. **Construcción de uniones:** Declara una unión que pueda interpretar 4 bytes como int, float o char[4]. Escribe un programa que guarde el entero 0x3F800000 e imprima el flotante resultante (debería ser 1.0 en IEEE 754).
4. **Gestión dinámica (Python vs C++):** Implementa una pila dinámica que crezca duplicando capacidad. Compara la complejidad temporal de push.
5. **Caso de estudio (endianness):** Recibes un archivo binario con números enteros de 32 bits en big-endian. Escribe una función en C que los convierta a little-endian para procesarlos en x86.

Guías de solución: Para el ejercicio 2, `p[2] == 'C'`. Al hacer `p++`, apunta a "BCD". Para el 3, recuerda que float y int comparten la misma dirección, pero la reinterpretación depende de la representación.

8. Glosario

- **Alineación (Alignment):** Restricción impuesta por el hardware que requiere que ciertos tipos de datos comiencen en direcciones múltiplo de su tamaño (ej. int de 4 bytes debe empezar en dirección múltiplo de 4).
- **Big-endian:** Formato de almacenamiento donde el byte más significativo de una palabra se guarda en la dirección de memoria más baja.
- **Caché (Cache):** Memoria de alta velocidad ubicada entre la CPU y la RAM, que almacena copias de datos e instrucciones usados frecuentemente para reducir el tiempo de acceso.
- **Desbordamiento de búfer (Buffer overflow):** Error que ocurre cuando se escribe más allá del límite de un arreglo o buffer, corrompiendo memoria adyacente.
- **Desbordamiento de pila (Stack overflow):** Condición en la que la pila supera su capacidad máxima, normalmente por recursión infinita o variables locales excesivamente grandes.
- **Dirección base (Base address):** Dirección de memoria donde comienza una estructura de datos, un arreglo o un segmento de programa.
- **DRAM (Dynamic RAM):** Tipo de RAM que almacena cada bit en un capacitor y requiere refresco periódico. Es más lenta y económica que la SRAM.

- **Empaquetamiento (Packing):** Técnica para eliminar los bytes de relleno (padding) de una estructura, generalmente mediante directivas como `#pragma pack`, ahorrando memoria a costa de rendimiento.
- **Endianness:** Orden en que los bytes de una palabra multibyte se almacenan en memoria (little-endian vs big-endian).
- **Fragmentación (Memory fragmentation):** Fenómeno en el que la memoria libre se divide en muchos bloques pequeños no contiguos, dificultando asignaciones grandes.
- **Fuga de memoria (Memory leak):** Pérdida de memoria dinámica que ocurre cuando se pierde la referencia a un bloque de memoria sin liberarlo, haciéndolo inutilizable hasta que el programa termina.
- **Heap overflow:** Escritura más allá de los límites de un bloque de memoria asignado en el montón, que puede corromper estructuras adyacentes de gestión de memoria.
- **Little-endian:** Formato de almacenamiento donde el byte menos significativo de una palabra se guarda en la dirección de memoria más baja (común en x86).
- **Memoria virtual:** Técnica de gestión de memoria que da a cada programa la ilusión de tener un espacio de direcciones contiguo y privado, mapeando direcciones virtuales a direcciones físicas mediante la MMU.
- **MMU (Memory Management Unit):** Componente de hardware que traduce direcciones virtuales a físicas y controla la protección de memoria.
- **Padding (Relleno):** Bytes adicionales insertados por el compilador entre miembros de una estructura para satisfacer restricciones de alineación, aumentando el tamaño total.
- **Palabra (Word):** Unidad de datos nativa del procesador, generalmente de 2, 4 u 8 bytes (16/32/64 bits).
- **Puntero colgante (Dangling pointer):** Puntero que sigue almacenando una dirección de memoria que ya ha sido liberada; acceder a él provoca comportamiento indefinido.
- **Recolector de basura (Garbage Collector, GC):** Mecanismo automático de gestión de memoria que libera objetos dinámicos que ya no son alcanzables. Usado en Java, Python, C#, entre otros.

- **Referencia (Reference):** En lenguajes de alto nivel, abstracción que actúa como un puntero seguro y sin aritmética, utilizada para acceder a objetos en el heap.
- **Registro (Register):** Celda de memoria interna del procesador, de velocidad extremadamente alta, usada para operandos y resultados inmediatos.
- **Segmento BSS (Block Started by Symbol):** Segmento de memoria que contiene variables globales y estáticas no inicializadas (o inicializadas a cero). El sistema operativo las pone a cero antes de ejecutar main().
- **Segmento de datos (Data segment):** Segmento que contiene variables globales y estáticas que ya tienen un valor inicial explícito en el código fuente.
- **Segmento de texto (Text segment):** Segmento de solo lectura que almacena el código máquina ejecutable del programa.
- **SRAM (Static RAM):** Tipo de RAM que utiliza flip-flops, más rápida y sin necesidad de refresco, pero más cara y de menor capacidad; usada en caché.
- **Stack frame:** Bloque de datos en la pila que contiene las variables locales, parámetros y dirección de retorno de una función durante su ejecución.
- **Tamaño de palabra (Word size):** Número de bits que el procesador puede procesar en una sola operación (ej. 32 o 64 bits).

Referencias bibliográficas

1. Deitel, P. J., & Deitel, H. M. (2017). *Java: cómo programar* (10.^a ed.). Pearson.
2. Joyanes Aguilar, L. (2008). *Fundamentos de programación: algoritmos, estructuras de datos y objetos* (4.^a ed.). McGraw-Hill.
3. Joyanes Aguilar, L. (2016). *Estructuras de datos en C++* (2.^a ed.). McGraw-Hill.
4. Stroustrup, B. (2013). *The C++ programming language* (4.^a ed.). Addison-Wesley.
5. Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2.^a ed.). Prentice Hall.
6. Lutz, M. (2013). *Learning Python* (5.^a ed.). O'Reilly Media.
7. Sedgewick, R., & Wayne, K. (2017). *Computer science: an interdisciplinary approach*. Addison-Wesley.
8. Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4.^a ed.). Pearson.
9. IEEE Computer Society. (2019). *IEEE Standard for Floating-Point Arithmetic (IEEE 754-2019)*. IEEE. DOI: 10.1109/IEEESTD.2019.8766229
10. ISO/IEC 14882:2020. (2020). *Programming languages — C++*. International Organization for Standardization.